**PROGRAMA DE INGENIERÍA DE SOFTWARE / TECNOLOGÍA EN DESARROLLO DE SOFTWARE**

ASIGNATURA: PRUEBAS Y CALIDAD DEL SOFTWARE (SEMESTRE VII)

**TALLER GRUPAL**

*Unidad 2: RAA1_U2_Prueba_calidad_software*


**Integrantes del Grupo:**

• Ruben Dario Bayona Galvis


**Docente / Tutor:**

**C**arlos Carrascal - Pruebas y Calidad de Software

**Fecha de Entrega:**

Noviembre de 2026

## Resumen

Se automatizaron 17 pruebas funcionales del módulo de compras de la tienda de demostración SauceDemo con Selenium WebDriver y JUnit 5. Las 17 pasaron, con una tasa de éxito del 100 % en 229,7 segundos y se alojo en el repositorio https://github.com/rubendario90/AA2-pruebas-saucedemo.git.

| Elemento | Detalle |
|---|---|
| Aplicación bajo prueba | SauceDemo (Swag Labs), tienda web de demostración |
| Módulos probados | Inicio y cierre de sesión, catálogo y carrito, proceso de compra (checkout) |
| Herramienta | Selenium WebDriver 4.27 + JUnit 5.11, con Maven 3.9.9 |
| Entorno | Windows 11, Java 17.0.13, Google Chrome 156 |
| Resultado | 17 pruebas, 0 fallos, 0 errores (`BUILD SUCCESS`) |

El trabajo lo realizó una sola persona. Donde la actividad pide reflexión grupal, se incluye una reflexión individual.

## Guía paso a paso

El trabajo siguió 5 pasos: preparar el entorno, analizar la aplicación, diseñar los casos, construir los scripts y ejecutarlos.

### Paso 1. Preparar el entorno

1. Instalar Java 17, Maven 3.9 y Google Chrome. Comprobar con `java -version` y `mvn -version`.
2. Crear el proyecto Maven `AA2-pruebas-saucedemo` con las dependencias `selenium-java 4.27.0` y `junit-jupiter 5.11.3`.
3. Agregar los plugins `maven-surefire-plugin` (ejecuta las pruebas) y `maven-surefire-report-plugin` (genera el reporte HTML).
4. No hace falta descargar ChromeDriver: Selenium Manager lo descarga automáticamente.

### Paso 2. Analizar la aplicación

Se recorrió SauceDemo a mano para identificar el flujo principal de compra:

1. Iniciar sesión con un usuario de prueba.
2. Ver y ordenar el catálogo de 6 productos.
3. Agregar productos al carrito.
4. Revisar el carrito.
5. Diligenciar los datos del comprador.

6. Revisar el resumen con subtotal, impuesto y total.
7. Finalizar la compra.

Con las herramientas de desarrollador del navegador (F12) se revisaron los elementos de cada pantalla. La aplicación marca sus elementos con el atributo `data-test`, así que se usaron localizadores como `[data-test='username']`. Estos localizadores son más estables que las clases CSS o los XPath largos, porque no cambian cuando cambia el diseño.

### Paso 3. Diseñar los casos de prueba

Para cada módulo se definieron casos positivos (el usuario hace todo bien) y negativos (datos faltantes o inválidos). Las técnicas aplicadas fueron:

- **Partición de equivalencia:** un usuario válido, uno bloqueado y una clave incorrecta representan cada clase de entrada.
- **Valores límite:** campos vacíos en el login y en el checkout.
- **Pruebas guiadas por datos:** el mismo script se ejecuta con varios juegos de datos, escritos en `@CsvSource` o en un archivo CSV externo.

### Paso 4. Construir los scripts con el patrón Page Object

El código se organizó en 3 capas para que sea reutilizable y fácil de mantener:

| Capa | Clases | Responsabilidad |
|---|---|---|
| Páginas | `BasePage`, `LoginPage`, `InventoryPage`, `CartPage`, `CheckoutPage` | Localizadores y acciones de cada pantalla. Es lo único que cambia si cambia la interfaz. |
| Base de pruebas | `BaseTest` | Abre y cierra Chrome, inicia sesión y guarda capturas de evidencia. |
| Pruebas | `LoginTest`, `InventoryTest`, `CheckoutTest` | Escenarios y verificaciones. No contienen localizadores. |
| Datos | `datos_checkout_invalido.csv` | Datos de prueba fuera del código. |

Buenas prácticas aplicadas:

- **Esperas explícitas** (`WebDriverWait`, 10 s) en lugar de pausas fijas como `Thread.sleep`.
- **Interfaz fluida:** cada acción devuelve la página siguiente, de modo que el flujo se lee como lo haría un usuario: `iniciarSesion().agregarAlCarrito(...).irAlCarrito().irAlCheckout()`.
- **Evidencias automáticas:** `tomarEvidencia(paso)` guarda capturas en `evidencias/<clase>/<caso>/`.
- **Identificadores de caso** (CP-01 a CP-10) en `@DisplayName`, para rastrear cada prueba en el reporte.
- **Modo sin ventana:** con `-Dheadless=true` las pruebas corren sin abrir Chrome, como en un servidor de integración continua.

### Paso 5. Ejecutar y revisar resultados

```
cd AA2-pruebas-saucedemo
mvn test                  # con ventana de Chrome visible
mvn test -Dheadless=true    # sin ventana
```

Cada ejecución produce 3 salidas:

- El resultado en la consola.
- El reporte HTML en `target/reports/surefire.html`.
- Las capturas en la carpeta `evidencias/`.

## Casos de prueba

Los 10 casos diseñados se convierten en 17 ejecuciones, porque los casos parametrizados corren una vez por cada juego de datos. Las 17 pasaron.

| ID | Módulo | Caso | Datos de entrada | Resultado esperado | Resultado |
|---|---|---|---|---|---|
| CP-01 | Login | Login válido | `standard_user` / `secret_sauce` | Abre `/inventory.html` con el título "Products" | Pasó |
| CP-02.1 | Login | Usuario bloqueado | `locked_out_user` / `secret_sauce` | "Sorry, this user has been locked out." | Pasó |
| CP-02.2 | Login | Clave incorrecta | `standard_user` / `claveMala` | "Username and password do not match…" | Pasó |
| CP-02.3 | Login | Usuario vacío | (vacío) / `secret_sauce` | "Username is required" | Pasó |
| CP-02.4 | Login | Clave vacía | `standard_user` / (vacío) | "Password is required" | Pasó |
| CP-03 | Login | Cerrar sesión | Menú → Logout | Regresa a la pantalla de login | Pasó |
| CP-04 | Catálogo | Cantidad de productos | — | Se muestran 6 productos | Pasó |
| CP-05.1 | Catálogo | Ordenar A → Z | Opción `az` | Nombres en orden ascendente | Pasó |
| CP-05.2 | Catálogo | Ordenar Z → A | Opción `za` | Nombres en orden descendente | Pasó |
| CP-06.1 | Catálogo | Precio menor a mayor | Opción `lohi` | Precios en orden ascendente | Pasó |
| CP-06.2 | Catálogo | Precio mayor | Opción `hilo` | Precios en orden | Pasó |

| ID | Módulo | Caso | Datos de entrada | Resultado esperado | Resultado |
|---|---|---|---|---|---|
| | | a menor | | descendente | |
| CP-07 | Carrito | Agregar y quitar | Backpack + Bike Light | El contador pasa por 2, 1 y 0 | Pasó |
| CP-08 | Carrito | Contenido del carrito | Backpack + Fleece Jacket | El carrito lista los 2 productos | Pasó |
| CP-09 | Checkout | Compra completa | Ana / Pérez / 110111 | Subtotal $39.98; total = subtotal + impuesto; "Thank you for your order!" | Pasó |
| CP-10.1 | Checkout | Falta el nombre | (vacío) / Pérez / 110111 | "Error: First Name is required" | Pasó |
| CP-10.2 | Checkout | Falta el apellido | Ana / (vacío) / 110111 | "Error: Last Name is required" | Pasó |
| CP-10.3 | Checkout | Falta el código postal | Ana / Pérez / (vacío) | "Error: Postal Code is required" | Pasó |

Los datos de CP-10 se leen del archivo externo `datos_checkout_invalido.csv`, así que se pueden agregar casos nuevos sin tocar el código:

```
nombre,apellido,postal,errorEsperado
,Pérez,110111,Error: First Name is required
Ana,,110111,Error: Last Name is required
Ana,Pérez,,Error: Postal Code is required
```

## Código de los scripts

Se incluyen las 4 clases que muestran la estructura. `LoginPage`, `CartPage`, `CheckoutPage`, `LoginTest` e `InventoryTest` siguen el mismo patrón y están en el proyecto entregado,y están disponibles en el repositorio del proyecto: https://github.com/tu-usuario/AA2-pruebas-saucedemo .

### `BasePage.java` – acciones comunes de todas las páginas

```
package saucedemo.pages;

import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.*;
import java.time.Duration;

/** Acciones comunes a todas las páginas: esperas explícitas, escribir, hacer clic,
leer texto. */
public abstract class BasePage {
    protected static final String URL_BASE = "https://www.saucedemo.com";
    protected final WebDriver driver;
```

```
    protected final WebDriverWait wait;

    private final By titulo      = By.cssSelector("[data-test='title']");
    private final By mensajeError = By.cssSelector("[data-test='error']");

    protected BasePage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    protected WebElement visible(By localizador) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(localizador));
    }

    protected void clic(By localizador) {
        wait.until(ExpectedConditions.elementToBeClickable(localizador)).click();
    }

    protected void escribir(By localizador, String texto) {
        WebElement campo = visible(localizador);
        campo.clear();
        campo.sendKeys(texto);
    }

    protected boolean existe(By localizador) {
        return !driver.findElements(localizador).isEmpty();
    }

    public String titulo()      { return visible(titulo).getText(); }
    public String mensajeError() { return visible(mensajeError).getText(); }
    public String urlActual()    { return driver.getCurrentUrl(); }
}
```

**`InventoryPage.java` – Page Object del catálogo**

```
package saucedemo.pages;

import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.Select;
import java.util.List;

/** Page Object del catálogo de productos. */
public class InventoryPage extends BasePage {
    private final By productos      = By.cssSelector("[data-test='inventory-item']");
    private final By nombres        = By.cssSelector("[data-test='inventory-item-
name']");
    private final By precios        = By.cssSelector("[data-test='inventory-item-
price']");
    private final By selectOrden    = By.cssSelector("[data-test='product-sort-
container']");
    private final By contadorCarrito = By.cssSelector("[data-test='shopping-cart-
badge']");
    private final By enlaceCarrito   = By.cssSelector("[data-test='shopping-cart-
link']");
    private final By botonMenu      = By.id("react-burger-menu-btn");
```

```
    private final By enlaceLogout    = By.cssSelector("[data-test='logout-sidebar-
link']");

    public InventoryPage(WebDriver driver) { super(driver); }

    public int cantidadProductos() {
        visible(productos);
        return driver.findElements(productos).size();
    }

    public List<String> nombresProductos() {
        visible(nombres);
        return
driver.findElements(nombres).stream().map(WebElement::getText).toList();
    }

    public List<Double> preciosProductos() {
        visible(precios);
        return driver.findElements(precios).stream()
                .map(e -> Double.parseDouble(e.getText().replace("$", "")))
                .toList();
    }

    /** Ordena el catálogo. Valores: az, za, lohi, hilo. */
    public InventoryPage ordenarPor(String valor) {
        new Select(visible(selectOrden)).selectByValue(valor);
        return this;
    }

    /** Busca el producto por su nombre visible y pulsa su botón "Add to cart". */
    public InventoryPage agregarAlCarrito(String nombreProducto) {
        botonDelProducto(nombreProducto, "add-to-cart").click();
        return this;
    }

    public InventoryPage quitarDelCarrito(String nombreProducto) {
        botonDelProducto(nombreProducto, "remove").click();
        return this;
    }

    /** Número que muestra el ícono del carrito; 0 si no hay contador. */
    public int contadorCarrito() {
        return existe(contadorCarrito)
                ? Integer.parseInt(driver.findElement(contadorCarrito).getText())
                : 0;
    }

    public CartPage irAlCarrito() {
        clic(enlaceCarrito);
        return new CartPage(driver);
    }

    public LoginPage cerrarSesion() {
        clic(botonMenu);
        clic(enlaceLogout);
```

```
        return new LoginPage(driver);
    }

    private WebElement botonDelProducto(String nombreProducto, String prefijoBoton) {
        visible(productos);
        return driver.findElements(productos).stream()
                .filter(p -> p.findElement(nombres).getText().equals(nombreProducto))
                .findFirst()
                .orElseThrow(() -> new AssertionError("No existe el producto: " +
nombreProducto))
                .findElement(By.cssSelector("button[data-test^='" + prefijoBoton +
"']"));
    }
}
```

**`BaseTest.java` – configuración común y evidencias**

```
package saucedemo.base;

import org.junit.jupiter.api.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import saucedemo.pages.InventoryPage;
import saucedemo.pages.LoginPage;
import java.nio.file.*;
import java.util.Map;

public abstract class BaseTest {
    protected static final String USUARIO_VALIDO = "standard_user";
    protected static final String CLAVE_VALIDA   = "secret_sauce";

    protected WebDriver driver;
    private Path carpetaEvidencias;

    @BeforeEach
    void iniciarNavegador(TestInfo info) {
        ChromeOptions opciones = new ChromeOptions();
        // Evita la ventana de Chrome "cambia tu contraseña", que tapa la página
        opciones.setExperimentalOption("prefs", Map.of(
                "credentials_enable_service", false,
                "profile.password_manager_enabled", false,
                "profile.password_manager_leak_detection", false));
        opciones.addArguments("--disable-features=PasswordLeakDetection", "--window-
size=1366,900");
        if (Boolean.getBoolean("headless")) opciones.addArguments("--headless=new");

        driver = new ChromeDriver(opciones);   // Selenium Manager gestiona el driver
        carpetaEvidencias = Paths.get("evidencias",
                info.getTestClass().map(Class::getSimpleName).orElse("General"),
                limpiar(info.getDisplayName()));
    }

    @AfterEach
    void cerrarNavegador() {
```

```
        tomarEvidencia("resultado_final");
        if (driver != null) driver.quit();
    }

    /** Atajo reutilizable: abre SauceDemo e inicia sesión con el usuario válido. */
    protected InventoryPage iniciarSesionValida() {
        return new LoginPage(driver).abrir().iniciarSesion(USUARIO_VALIDO,
CLAVE_VALIDA);
    }

    /** Guarda una captura del navegador con el nombre del paso indicado. */
    protected void tomarEvidencia(String paso) {
        try {
            Files.createDirectories(carpetaEvidencias);
            Path destino = carpetaEvidencias.resolve(limpiar(paso) + ".png");
            Path origen = ((TakesScreenshot)
driver).getScreenshotAs(OutputType.FILE).toPath();
            Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);
        } catch (Exception e) {
            System.err.println("No se pudo guardar la captura: " + e.getMessage());
        }
    }

    private static String limpiar(String texto) {
        return texto.replaceAll("[^a-zA-Z0-9-_]", "_");
    }
}
```

**`CheckoutTest.java` – pruebas del proceso de compra**

```
package saucedemo.tests;

import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvFileSource;
import saucedemo.base.BaseTest;
import saucedemo.pages.*;
import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Módulo 3 - Proceso de compra (checkout)")
class CheckoutTest extends BaseTest {

    @Test
    @DisplayName("CP-09 Compra completa de principio a fin")
    void compraCompleta() {
        CheckoutPage checkout = iniciarSesionValida()
                .agregarAlCarrito("Sauce Labs Backpack")      // $29.99
                .agregarAlCarrito("Sauce Labs Bike Light")    // $9.99
                .irAlCarrito()
                .irAlCheckout()
                .diligenciarDatos("Ana", "Pérez", "110111");
        tomarEvidencia("1_datos_comprador");

        checkout.continuar();
        tomarEvidencia("2_resumen_orden");
```

```
        assertAll("Resumen de la orden",
            () -> assertEquals(39.98, checkout.subtotal(), 0.001, "Subtotal
incorrecto"),
            () -> assertEquals(checkout.subtotal() + checkout.impuesto(),
checkout.total(), 0.01,
                    "El total no es subtotal + impuesto")
        );

        checkout.finalizar();
        assertEquals("Thank you for your order!", checkout.mensajeConfirmacion());
    }

    @ParameterizedTest(name = "CP-10.{index} nombre=''{0}'' apellido=''{1}''
postal=''{2}''")
    @CsvFileSource(resources = "/datos_checkout_invalido.csv", numLinesToSkip = 1)
    @DisplayName("CP-10 Checkout con datos incompletos muestra el error correcto")
    void checkoutDatosIncompletos(String nombre, String apellido, String postal,
String errorEsperado) {
        CheckoutPage checkout = iniciarSesionValida()
                .agregarAlCarrito("Sauce Labs Onesie")
                .irAlCarrito()
                .irAlCheckout()
                .diligenciarDatos(vacioSiNulo(nombre), vacioSiNulo(apellido),
vacioSiNulo(postal))
                .continuar();

        assertEquals(errorEsperado, checkout.mensajeError());
    }

    /** En el CSV una celda vacía llega como null; el formulario necesita texto vacío.
*/
    private static String vacioSiNulo(String valor) {
        return valor == null ? "" : valor;
    }
}
```

## Evidencias

Las figuras 1 y 2 muestran el resultado global de la ejecución. Las capturas de las figuras 3 a 12 las tomó el propio script mientras ejecutaba cada caso.

**Figura 1.** Salida de `mvn test` en la terminal: 17 pruebas, 0 fallos, `BUILD SUCCESS`.

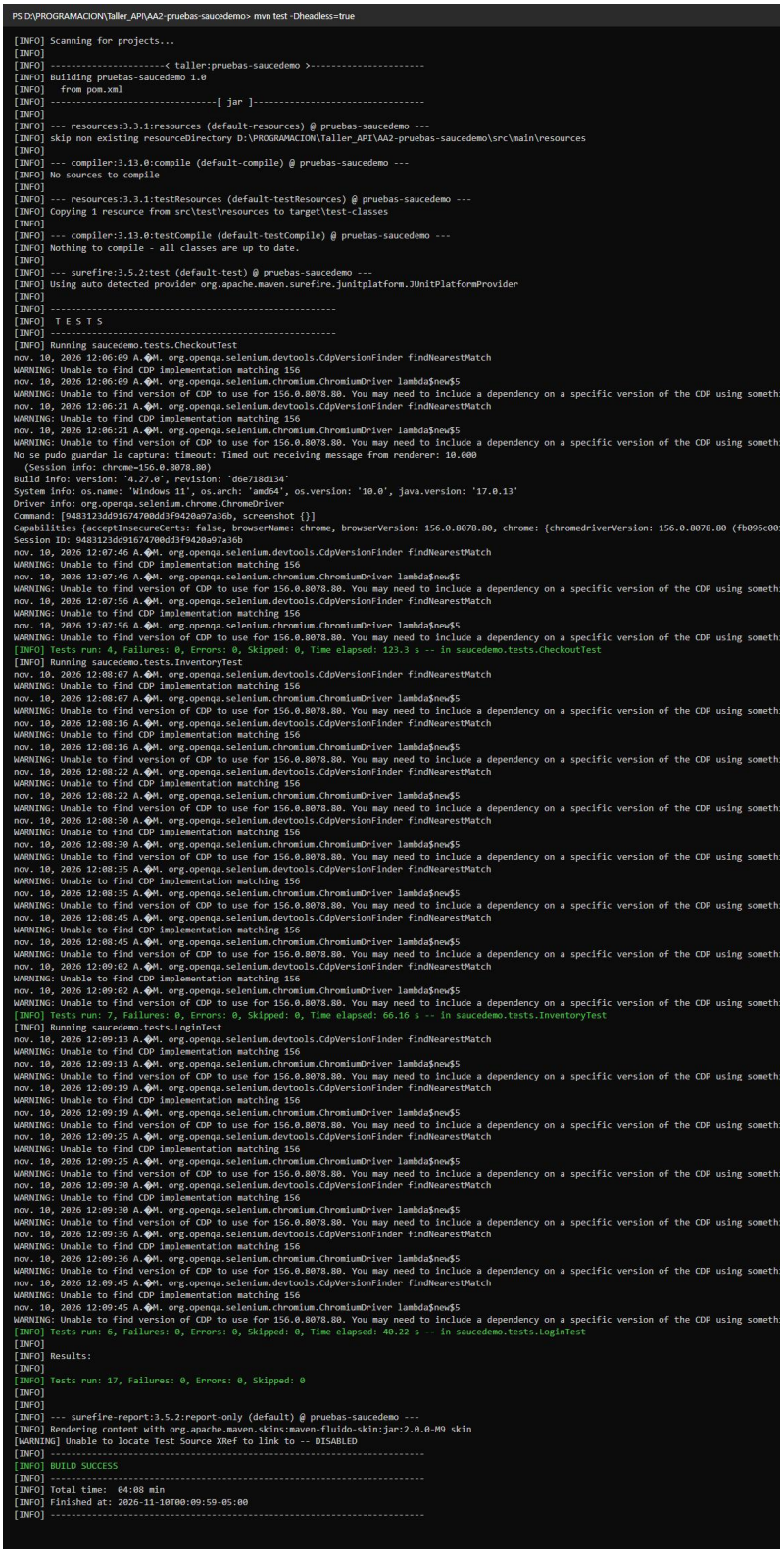



**Figura 2.** Reporte HTML de Surefire: 17 pruebas en 3 clases, 100 % de éxito.

# pruebas-saucedemo

Last Published: 2026-11-10 | Version: 1.0 pruebas-saucedemo

# Surefire Report

## Summary

[Summary] [Package List] [Test Cases]

| Tests | Errors | Failures | Skipped | Success Rate | Time |
|---|---|---|---|---|---|
| 17 | 0 | 0 | 0 | 100% | 229.7 s |

Note: failures are anticipated and checked for with assertions while errors are unanticipated.

## Package List

[Summary] [Package List] [Test Cases]

| Package | Tests | Errors | Failures | Skipped | Success Rate | Time |
|---|---|---|---|---|---|---|
| saucedemo.tests | 17 | 0 | 0 | 0 | 100% | 229.7 s |

Note: package statistics are not computed recursively, they only sum up all of its testsuites numbers.

### saucedemo.tests

| - | Class | Tests | Errors | Failures | Skipped | Success Rate | Time |
|---|---|---|---|---|---|---|---|
| | CheckoutTest | 4 | 0 | 0 | 0 | 100% | 123.3 s |
| | InventoryTest | 7 | 0 | 0 | 0 | 100% | 66.16 s |
| | LoginTest | 6 | 0 | 0 | 0 | 100% | 40.22 s |

## Test Cases

[Summary] [Package List] [Test Cases]

### CheckoutTest

| | | |
|---|---|---|
| | compraCompleta | 14.96 s |
| | checkoutDatosIncompletos(String, String, String, String)[1] | 85.56 s |
| | checkoutDatosIncompletos(String, String, String, String)[2] | 10.99 s |
| | checkoutDatosIncompletos(String, String, String, String)[3] | 11.41 s |

### InventoryTest

| | | |
|---|---|---|
| | carritoContieneProductos | 8.689 s |
| | ordenarPorNombre(String, boolean)[1] | 6.305 s |
| | ordenarPorNombre(String, boolean)[2] | 8.080 s |
| | ordenarPorPrecio(String, boolean)[1] | 5.647 s |
| | ordenarPorPrecio(String, boolean)[2] | 9.036 s |
| | agregarYQuitarProductos | 15.97 s |

**Figura 3.** CP-01: formulario de login diligenciado por el script antes de enviarlo.

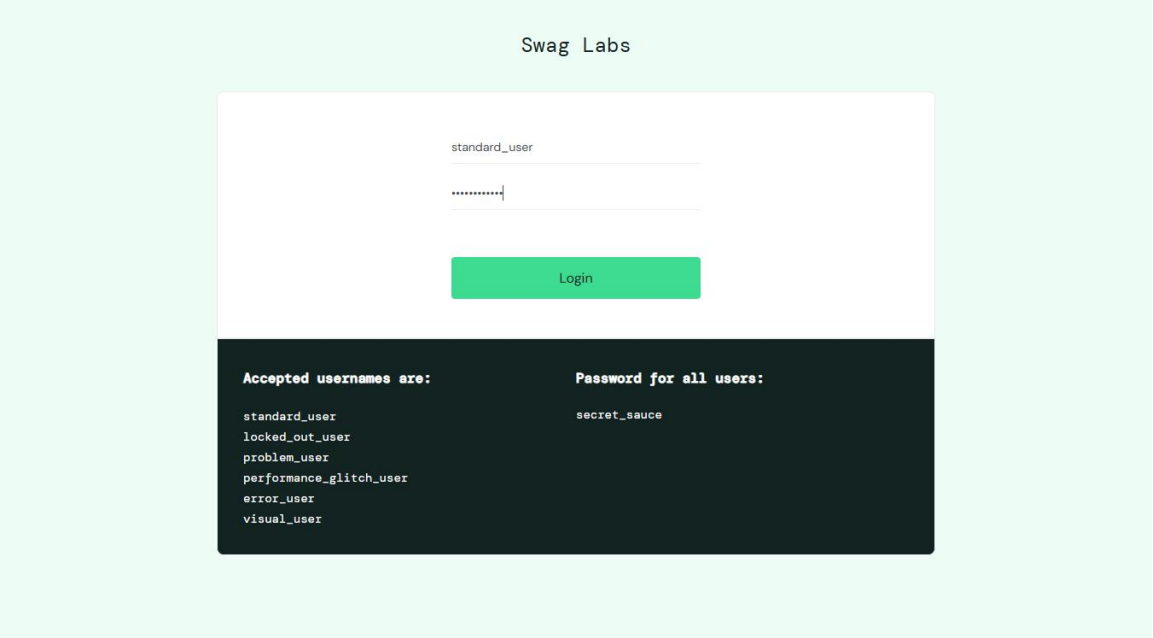



**Figura 4.** CP-01: login exitoso, se muestra el catálogo "Products".

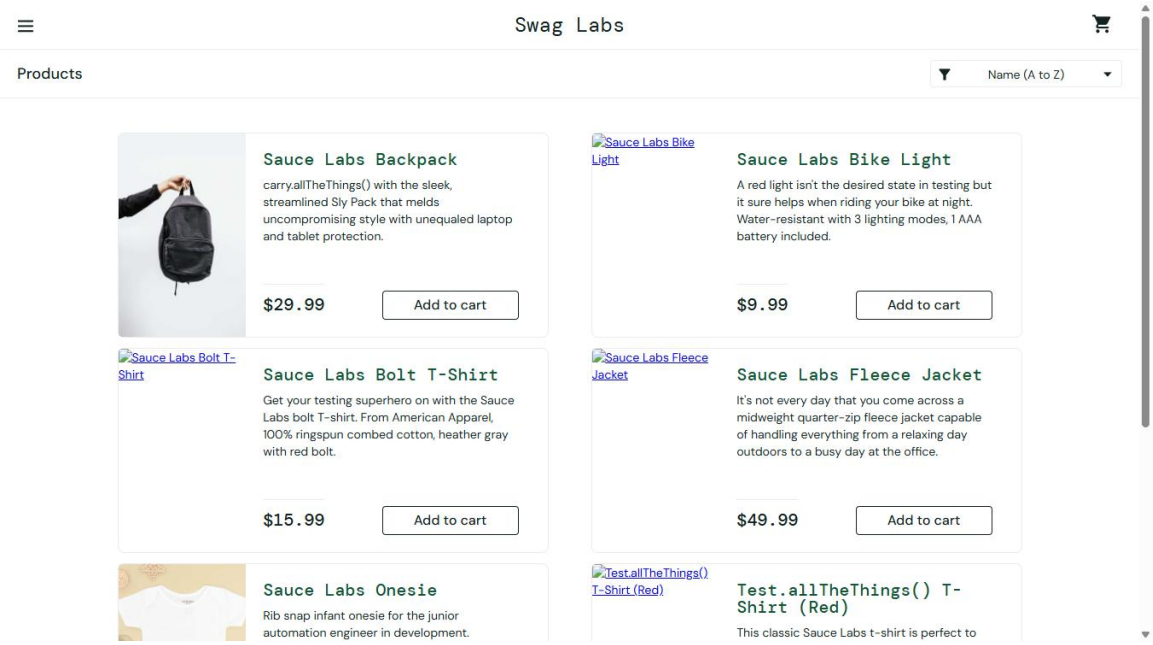


**Figura 5.** CP-02.1: el usuario bloqueado recibe el mensaje de error.

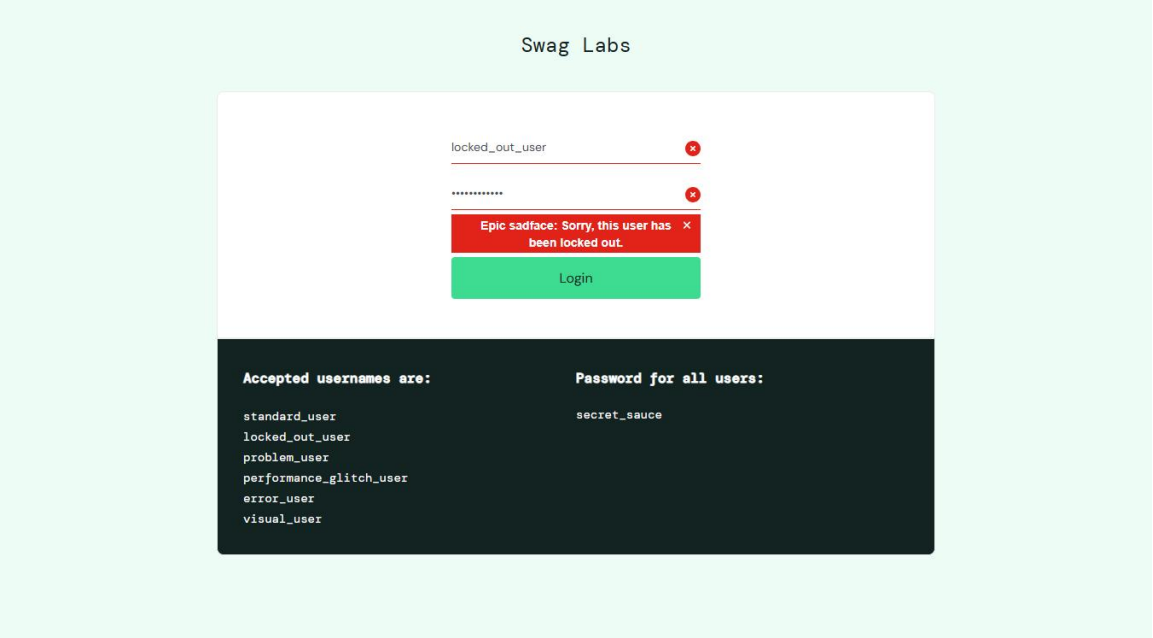



**Figura 6.** CP-06.1: catálogo ordenado por precio, de menor a mayor.

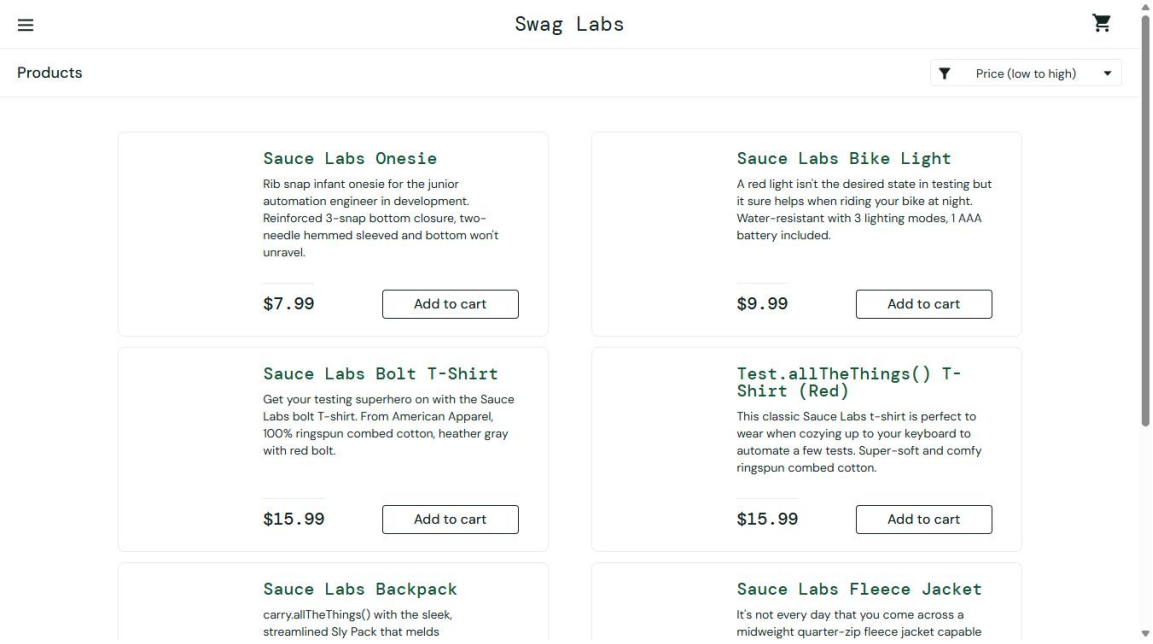


**Figura 7.** CP-07: el contador del carrito marca 2 después de agregar dos productos.

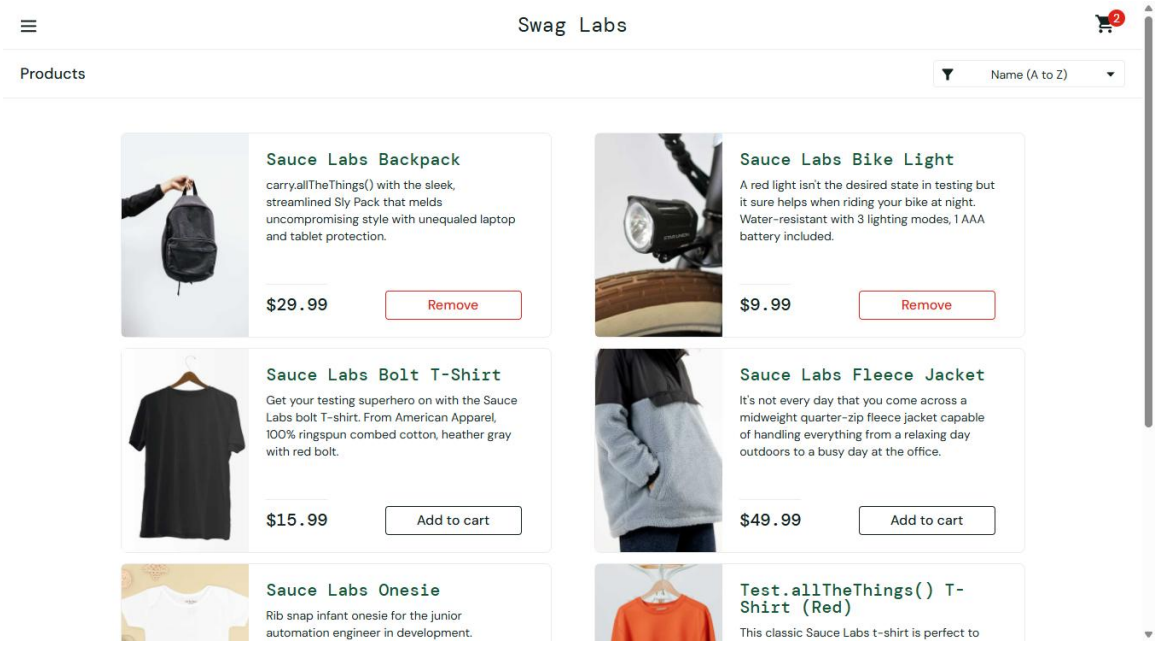



**Figura 8.** CP-08: el carrito lista los productos agregados.

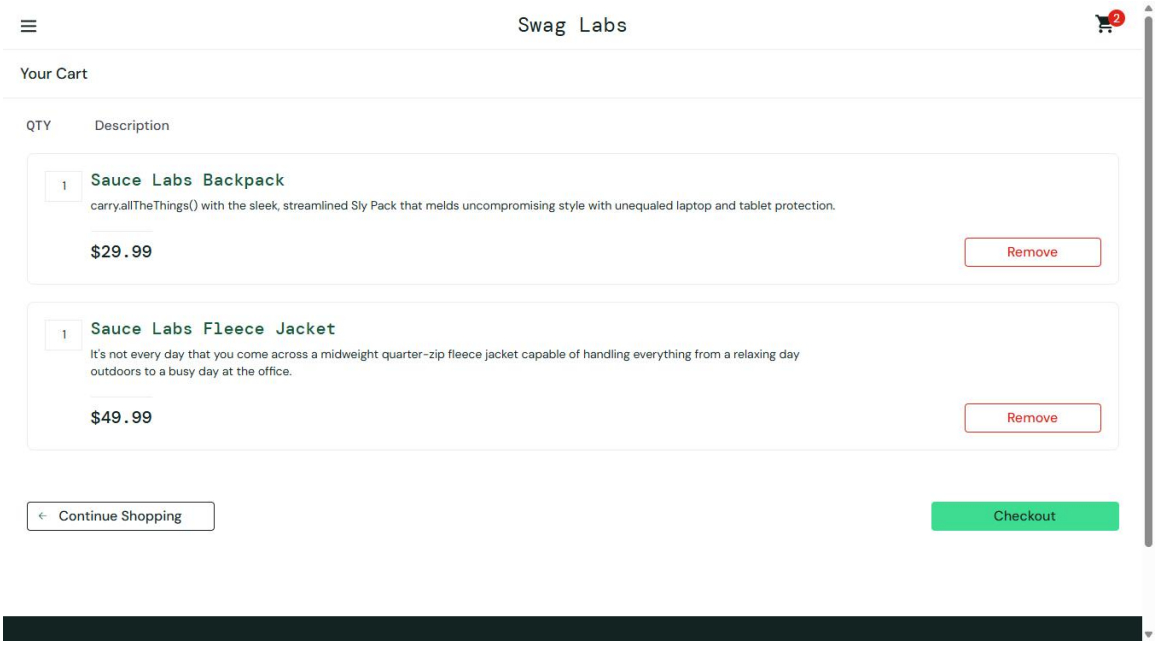


**Figura 9.** CP-09: datos del comprador diligenciados en el checkout.

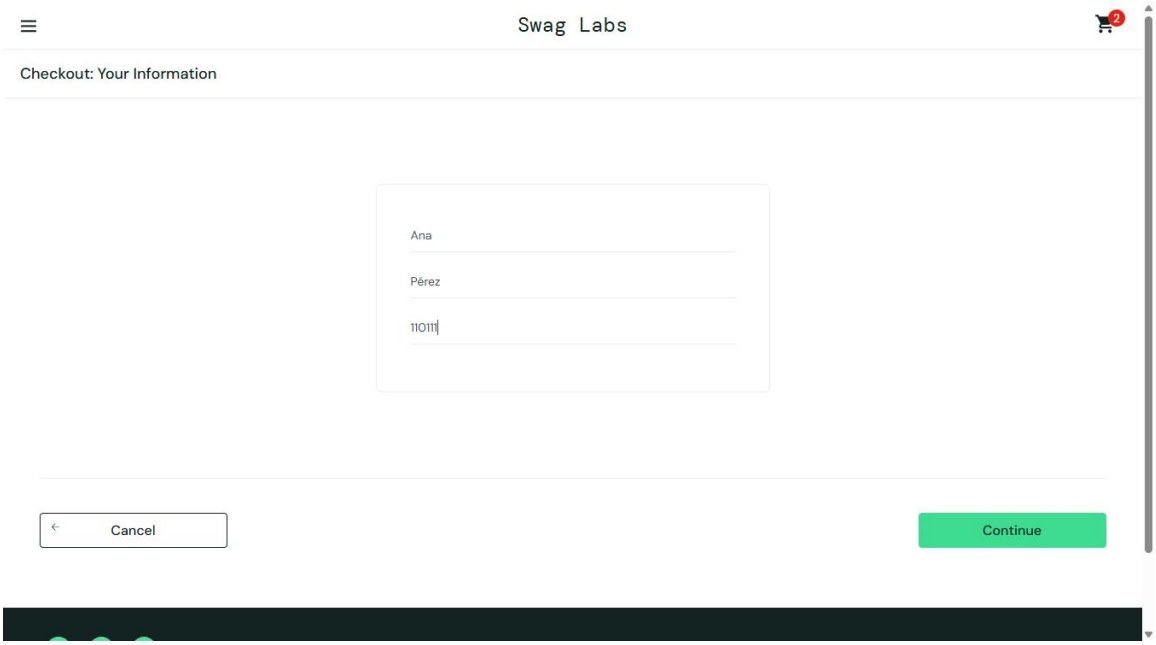



**Figura 10.** CP-09: resumen de la orden, con subtotal de $39.98.

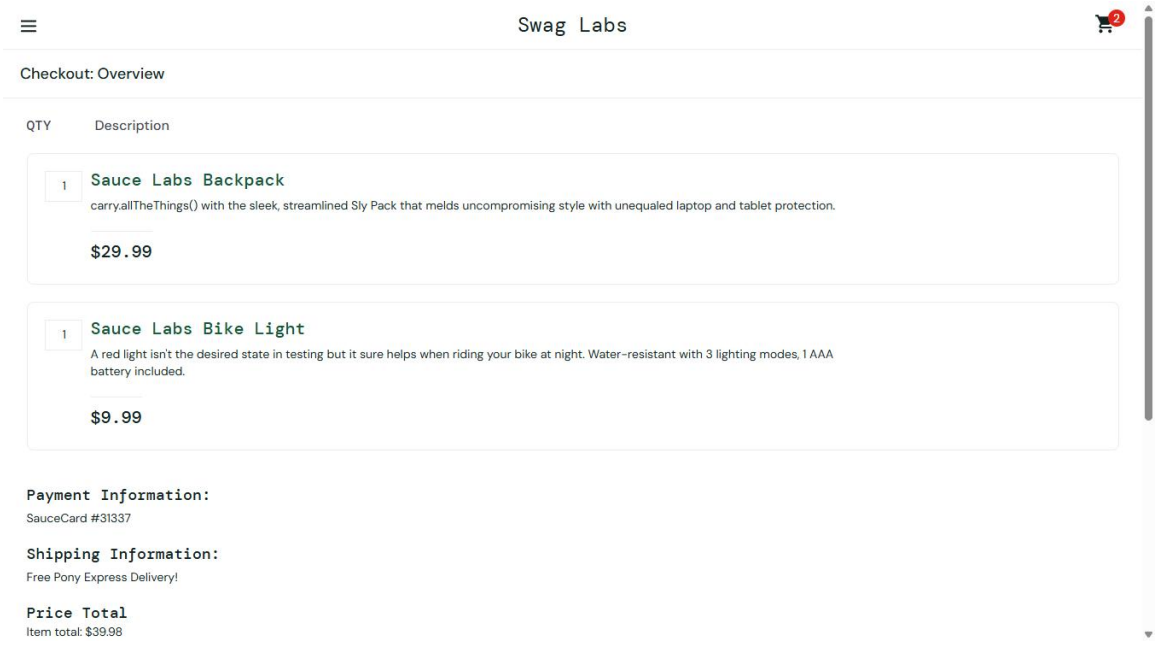


**Figura 11.** CP-09: compra confirmada con "Thank you for your order!".

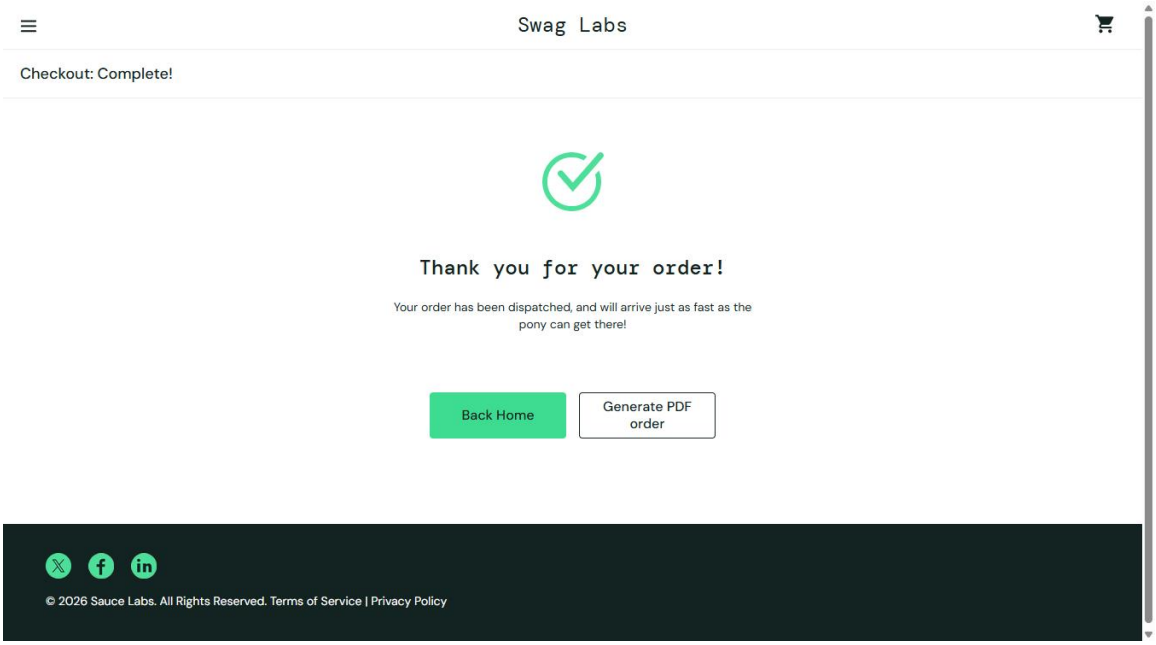
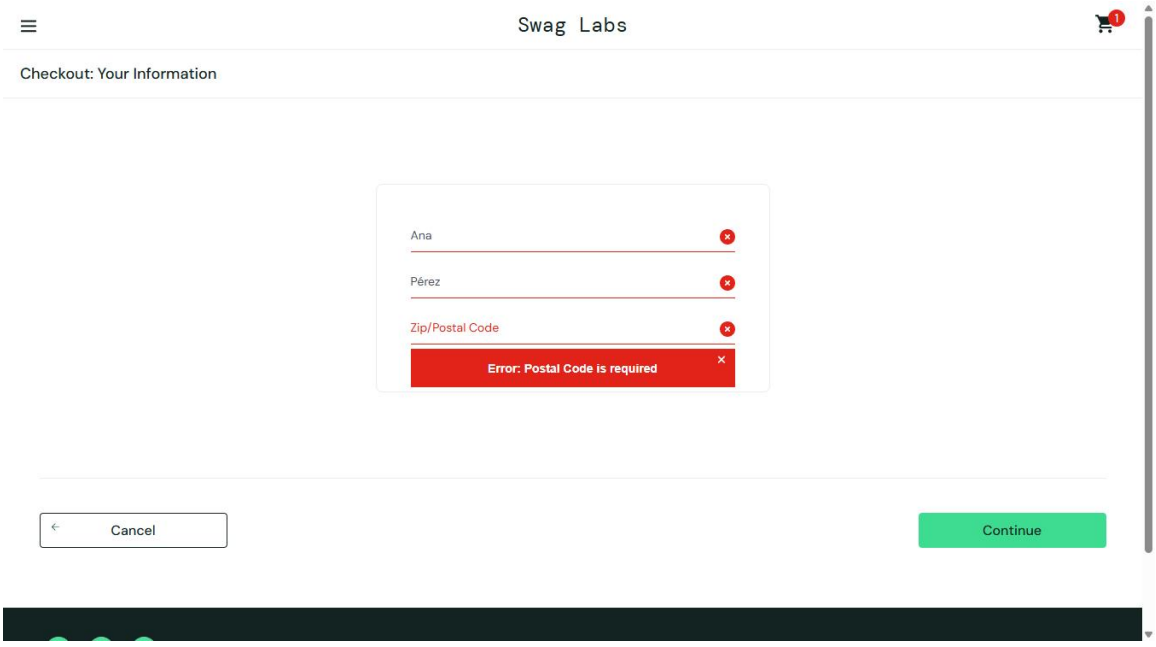



**Figura 12.** CP-10.3: el checkout sin código postal muestra el error.



## Hallazgos durante el desarrollo

1. **Dato de prueba mal escrito.** En la primera ejecución falló CP-02.1: se esperaba "Epic sadface: Sorry" y la aplicación mostró el mensaje completo. La coma dentro del mensaje hizo que `@CsvSource` lo partiera en dos columnas. Se corrigió poniendo el texto entre comillas simples. El error estaba en la prueba, no en la aplicación; revisar primero la prueba evita reportar defectos falsos.
2. **Fallos del entorno, no de la aplicación.** En dos ejecuciones, Chrome se cerró o dejó de responder (`SessionNotCreatedException` y `TimeoutException`). Al repetir la ejecución, todo pasó. Estas pruebas "intermitentes" (*flaky*) son un riesgo real de la automatización con navegador.
3. **Ventana emergente de Chrome.** Chrome muestra un aviso de "contraseña filtrada" al iniciar sesión con `secret_sauce`, y ese aviso puede tapar la página. Se desactivó con opciones de `ChromeOptions` en `BaseTest`.
4. **Sin defectos funcionales.** Con el usuario `standard_user`, SauceDemo se comportó como se esperaba en los 17 casos.

**Reflexión**

¿Qué utilidad tienen las pruebas automatizadas frente a las manuales?

La mayor utilidad es que puedo repetir las pruebas sin esfuerzo. Las 17 pruebas se ejecutan con un solo comando, mvn test, y terminan en menos de 4 minutos (229,7 segundos) sin que yo tenga que tocar el teclado. Hacer lo mismo a mano exige iniciar sesión, agregar productos, llenar el checkout y revisar mensajes en cada caso. Calculo que tardaría bastante más de media hora cada vez, y con el riesgo de escribir mal un dato o saltarme una verificación por cansancio.

Esto es más valioso después de cada cambio en la aplicación. Una prueba manual se hace una vez y se olvida, pero una automatizada queda como prueba de regresión que se puede ejecutar todas las veces que haga falta. Además, el script siempre verifica lo mismo con la misma precisión, como que el total sea exactamente subtotal más impuesto. También deja evidencia ordenada en capturas y en un reporte HTML, sin que tenga que acordarme de tomarla.

¿Qué dificultades encontré y cómo las resolví?

La primera fue el fallo de CP-02.1 en la primera ejecución. Al principio parecía que la aplicación mostraba un mensaje distinto al esperado, pero al revisar vi que la coma dentro del texto “Epic sadface: Sorry, this user has been locked out.” hacía que @CsvSource lo partiera en dos columnas. Lo resolví poniendo el texto entre comillas simples. Aprendí que, cuando una prueba falla, primero hay que revisar la prueba misma, porque el error puede estar en los datos y no en la aplicación.

La segunda fueron los cierres inesperados de Chrome, con SessionNotCreatedException y TimeoutException. Al repetir la ejecución todo pasó, lo que confirmó que eran problemas del entorno y no defectos de SauceDemo. La tercera fue la ventana de Chrome que avisa de una “contraseña filtrada” al usar secret_sauce, que podía tapar la página. La desactivé con ChromeOptions en BaseTest, así que la solución quedó en un solo lugar y aplica a todas las pruebas.

¿Por qué el patrón Page Object facilita el mantenimiento?

Page Object separa el “cómo” del “qué”. Los localizadores y las acciones de cada pantalla viven en su clase de página (LoginPage, InventoryPage, CartPage, CheckoutPage), y las pruebas solo describen el escenario y las verificaciones. Si cambia la interfaz, solo hay que tocar la clase de esa pantalla, y las pruebas siguen igual.

Por ejemplo, si cambiara el botón de login y los localizadores estuvieran repetidos en cada prueba, tendría que corregir el mismo localizador en muchos archivos. Además, las 17 pruebas pasan por el login, ya sea directamente o con iniciarSesionValida(), así que un solo cambio en la interfaz rompería toda la suite, y sería fácil olvidar alguna copia. Con Page Object basta con cambiar una línea en LoginPage. A eso se suma que la interfaz fluida hace que las pruebas se lean como los pasos de un usuario, y eso las hace más fáciles de entender y revisar.

¿Qué límites tiene la automatización?

Un límite claro son las pruebas intermitentes (flaky). Una prueba puede fallar por razones ajenas a la aplicación, como un navegador que no arranca, una red lenta o un tiempo de espera insuficiente. Eso obliga a analizar cada fallo antes de reportarlo, y si pasa seguido, le quita confianza a la suite. Además, como SauceDemo es un sitio externo, su disponibilidad no depende de mí.

Otro límite es que una prueba automatizada solo verifica lo que se le pidió. Puede confirmar que aparece “Thank you for your order!”, pero no puede juzgar si el diseño se ve bien, si los colores son legibles o si la experiencia es cómoda para el usuario. Tampoco detecta problemas que no están en sus aserciones; por ejemplo, mis pruebas no revisan que las imágenes de los productos sean las correctas. Por eso la automatización complementa la prueba manual y exploratoria, pero no la reemplaza.

¿Qué mejoraría?

Primero, probaría otros usuarios de SauceDemo, como problem_user, que muestra errores en imágenes y formularios, y performance_glitch_user, que responde con lentitud. Así la suite buscaría defectos y no solo confirmaría el flujo correcto, y también pondría a prueba que las esperas explícitas resisten la lentitud. También ejecutaría las pruebas en otros navegadores, como Firefox y Edge, para verificar la compatibilidad.

Segundo, integraría las pruebas en un pipeline de integración continua, por ejemplo con GitHub Actions. El proyecto ya lo permite gracias al modo -Dheadless=true, así que las pruebas podrían ejecutarse en cada cambio y publicar el reporte de forma automática. Para las pruebas intermitentes, agregaría reintentos controlados y registraría cuáles fallan con más frecuencia. Así podría distinguir los fallos del entorno de los defectos reales.

**Conclusión**

Se automatizó con éxito el flujo completo de compra de SauceDemo, desde el inicio de sesión hasta la confirmación del pedido. Las 17 pruebas funcionales pasaron, con una tasa de éxito del 100 %. El patrón Page Object, las esperas explícitas y las pruebas guiadas por datos permitieron cubrir los 10 casos diseñados con código reutilizable y fácil de mantener. Los datos de prueba quedaron en un archivo CSV externo, así que se pueden agregar casos nuevos sin modificar el código.

Durante el desarrollo, cada fallo se analizó hasta encontrar su causa: un dato mal escrito en la prueba y dos problemas del entorno, todos resueltos. Ninguno resultó ser un defecto de la aplicación. Esto confirma que, antes de reportar un error, hay que descartar primero la prueba y el entorno. El resultado es una suite estable, documentada y con evidencias automáticas, lista para usarse como prueba de regresión.